

Functional Testing

Complete Interview Questions & Answers

Beginner · Intermediate · Advanced · Scenario-Based

8 Topics

70+ Questions

All Levels

Detailed Answers

Table of Contents

1. Fundamentals of Functional Testing (Beginner)
2. Types of Functional Testing (Beginner)
3. Test Case Design & Techniques (Intermediate)
4. Test Planning & Strategy (Intermediate)
5. Defect / Bug Management (Intermediate)
6. Test Execution & Reporting (Intermediate)
7. Automation in Functional Testing (Advanced)
8. Scenario-Based & Situational Questions (Advanced)

Section 1 – Fundamentals of Functional Testing

Q1.

What is Functional Testing?

Functional Testing is a type of software testing that validates the software system against functional requirements and specifications. It focuses on WHAT the system does — verifying that each function of the application works in accordance with the requirement document.

The primary goal is to check the functionality of each feature by providing appropriate input, executing the function, and verifying the output against expected results.

Q2.

What is the difference between Functional and Non-Functional Testing?

Functional Testing:

- Tests WHAT the system does (features and functions)
- Verifies business requirements and user requirements
- Examples: Unit, Integration, System, UAT, Regression testing

Non-Functional Testing:

- Tests HOW the system performs (quality attributes)
 - Covers performance, security, usability, scalability
 - Examples: Load, Stress, Performance, Security, Usability testing
-

Q3.

What are the objectives of Functional Testing?

- Verify that the software behaves according to specified requirements
 - Ensure that all the functions of the application are working correctly
 - Validate the business processes end-to-end
 - Detect defects before the product reaches the end user
 - Verify usability and accessibility of the application
 - Ensure backward and forward compatibility where applicable
-

Q4.

What is a Test Case? What are its components?

A Test Case is a set of conditions or variables under which a tester determines whether a system under test satisfies requirements or works correctly.

Components of a Test Case:

- Test Case ID – Unique identifier
 - Test Case Title / Name – Brief description of what is being tested
 - Pre-conditions – Conditions that must be met before executing the test
 - Test Steps – Step-by-step actions to perform
 - Test Data – Input values used during execution
 - Expected Result – The anticipated output
 - Actual Result – The actual output after execution
 - Post-conditions – State of the system after the test
 - Pass/Fail Status – Whether test passed or failed
 - Priority / Severity – Importance level
-

Q5.

What is the difference between a Test Case and a Test Scenario?

Test Scenario:

- High-level description of what to test
- One-liner that describes a possible user action or goal
- Example: 'Verify login with valid credentials'

Test Case:

- Detailed step-by-step procedure derived from a test scenario
- Includes specific inputs, steps, expected outputs
- Example: Enter username 'admin', password 'Admin@123', click Login, verify dashboard loads

One test scenario can have multiple test cases covering different combinations.

Q6.

What is the Testing Life Cycle (STLC)?

The Software Testing Life Cycle (STLC) is a sequence of specific activities conducted during the testing process:

- 1. Requirement Analysis – Understand and analyse testable requirements
 - 2. Test Planning – Define scope, strategy, resources, and schedule
 - 3. Test Case Development – Design and document test cases and test data
 - 4. Test Environment Setup – Prepare hardware, software, and test data
 - 5. Test Execution – Execute test cases and log defects
 - 6. Test Closure – Evaluate completion criteria, prepare summary reports
-

Q7.

What is the difference between Verification and Validation?

Verification (Are we building the product right?):

- Static testing — reviews, walkthroughs, inspections
- Ensures the product is built according to specifications
- Done without executing code

Validation (Are we building the right product?):

- Dynamic testing — actual execution of the software
 - Ensures the product meets the end user's actual needs
 - Done by executing the application
-

Q8.

What is the difference between Quality Assurance (QA) and Quality Control (QC)?

Quality Assurance (QA):

- Process-oriented — focuses on preventing defects
- Proactive approach — improving the development process
- Examples: Process reviews, audits, standard definitions

Quality Control (QC):

- Product-oriented — focuses on identifying defects in the product
 - Reactive approach — finding bugs after the software is built
 - Examples: Test execution, defect reporting, code reviews
-

Q9.

What is Exploratory Testing?

Exploratory Testing is a type of functional testing where testers simultaneously design and execute tests without predefined test cases. The tester explores the application, learns it in real time, and uses that knowledge to design new tests.

- No pre-written scripts — tests are improvised on the fly
- Requires experience, intuition, and creativity
- Best suited for finding unexpected bugs missed by scripted tests
- Time-boxed sessions (called charter-based testing)

Tip: Mention exploratory testing when asked about testing with limited documentation.

Q10.

What is Ad Hoc Testing?

Ad Hoc Testing is an informal type of testing performed without any planning, documentation, or test cases. It is done randomly to break the system in unexpected ways.

- No formal process or structure
 - Relies on the tester's experience and intuition
 - Difficult to reproduce defects found
 - Difference from Exploratory: Ad hoc is entirely random; exploratory has some structure through charters
-

Section 2 – Types of Functional Testing

Q11.

What are the different types of Functional Testing?

- Unit Testing – Testing individual components or functions in isolation
 - Integration Testing – Testing interaction between integrated modules
 - System Testing – Testing the complete integrated system
 - Smoke Testing – Quick check to verify basic functionality after a build
 - Sanity Testing – Narrow regression testing after minor fixes
 - Regression Testing – Re-testing after changes to ensure no new bugs
 - User Acceptance Testing (UAT) – End-user validation of the system
 - Alpha Testing – Testing by internal team before release
 - Beta Testing – Testing by real users in a production-like environment
 - End-to-End Testing – Testing complete workflows from start to finish
-

Q12.

What is the difference between Smoke Testing and Sanity Testing?

Smoke Testing:

- Also called 'Build Verification Testing'
- Tests the most critical functionalities of the entire application
- Performed after every new build is received
- Broad but shallow — checks if the build is stable enough for further testing
- If smoke tests fail, the build is rejected

Sanity Testing:

- Subset of regression testing performed after minor bug fixes
 - Narrow and deep — focuses on specific functionality
 - Verifies that the fix works and has not broken related features
 - Performed without a test script (usually undocumented)
-

Q13.

What is Regression Testing and when is it performed?

Regression Testing is re-testing a previously tested application after code changes (bug fixes, new features, or enhancements) to ensure that existing functionalities still work correctly.

When to perform:

- After a bug fix is applied
- After adding a new feature
- After code refactoring
- After environment or configuration changes
- Before a software release

Types of Regression Testing:

- Full Regression – Re-run all test cases
 - Partial/Selective Regression – Only related test cases
 - Unit Regression – Test only the changed unit
-

Q14.

What is User Acceptance Testing (UAT)?

UAT is the final phase of testing where the software is tested by the actual end users or business stakeholders to validate that it meets business requirements and is ready for production.

- Conducted by clients, end users, or business analysts
 - Uses real-world scenarios and business cases
 - Verifies the system against user requirements, not technical specs
 - Two types: Alpha (in-house) and Beta (external users)
 - Successful UAT leads to sign-off and production deployment
-

Q15.

What is Integration Testing? What are its approaches?

Integration Testing verifies that different modules or components work together correctly when combined.

Approaches:

- Big Bang – All modules integrated and tested at once
 - Top-Down – Testing starts from top-level modules, stubs replace lower modules
 - Bottom-Up – Testing starts from lower modules, drivers replace upper modules
 - Sandwich / Hybrid – Combination of top-down and bottom-up
 - Incremental – Modules integrated one at a time and tested
-

Q16.

What is System Testing?

System Testing is a level of testing where a complete and integrated software system is tested to evaluate the system's compliance with its specified requirements. It is performed by an independent testing team.

- Tests the entire application as a whole
 - Black-box testing approach — no knowledge of internal code
 - Covers both functional and non-functional requirements
 - Performed in an environment that mirrors production
-

Q17.

What is End-to-End (E2E) Testing?

End-to-End testing validates the entire software workflow from start to finish, simulating real user scenarios to ensure the system integrates correctly across all layers (UI, database, APIs, etc.).

- Tests complete business flows, not individual features
 - Example: Login → Add to cart → Checkout → Payment → Order confirmation
 - Validates data integrity across all system components
 - Time-consuming but provides the highest confidence
-

Section 3 – Test Case Design & Techniques

Q18.

What are the different test case design techniques?

- Equivalence Partitioning (EP)
 - Boundary Value Analysis (BVA)
 - Decision Table Testing
 - State Transition Testing
 - Use Case Testing
 - Error Guessing
 - Cause-Effect Graphing
 - Pairwise / All-Pairs Testing
-

Q19.

What is Equivalence Partitioning?

Equivalence Partitioning (EP) is a test design technique that divides input data into equivalent partitions, where each partition is expected to be processed in the same way. Only one value per partition needs to be tested.

Example — Age field accepts 18–60:

- Valid partition: 18–60 (test with e.g. 30)
- Invalid partition 1: Less than 18 (test with e.g. 10)
- Invalid partition 2: Greater than 60 (test with e.g. 70)

This reduces the total number of test cases while maintaining good coverage.

Q20.

What is Boundary Value Analysis (BVA)?

BVA is a technique that focuses on testing the boundary values of input ranges, since defects are more likely to occur at the edges of valid partitions.

For a field accepting values 1–100, test:

- Minimum boundary: 1
- Just below minimum: 0
- Just above minimum: 2
- Maximum boundary: 100
- Just above maximum: 101
- Just below maximum: 99

Tip: BVA is often used together with Equivalence Partitioning for comprehensive coverage.

Q21.

What is Decision Table Testing?

Decision Table Testing is a technique used when the system's behaviour depends on combinations of conditions. It creates a table mapping all combinations of inputs (conditions) to their expected outputs (actions).

Example — Loan Approval:

- Conditions: Credit Score (Good/Bad), Income (High/Low), Employment (Yes/No)
- Each combination results in a different decision (Approve/Reject)

Ensures all possible condition combinations are tested, especially useful for business rules.

Q22.

What is State Transition Testing?

State Transition Testing is used when the system can exist in different states and transitions between them based on events or inputs. A state transition diagram is used to derive test cases.

Example — ATM machine states:

- Idle → Card Inserted (event: insert card)
- Card Inserted → PIN Verified (event: correct PIN)
- PIN Verified → Transaction (event: select transaction)
- Transaction → Idle (event: eject card)

Test cases cover valid transitions, invalid transitions, and boundary states.

Q23.

What is Cause-Effect Graphing?

Cause-Effect Graphing is a test design technique that analyses the logical relationships between causes (inputs) and effects (outputs). It uses a Boolean graph to model these relationships and then derives test cases from the graph.

- Causes represent input conditions
 - Effects represent output actions or system behaviours
 - Logical operators (AND, OR, NOT) define the relationships
 - Useful for complex business logic with many interdependencies
-

Q24.

What is Pairwise (All-Pairs) Testing?

Pairwise Testing is a combinatorial technique where test cases are designed to cover all possible pairs of input combinations. Since most defects are caused by interactions between two variables, testing all pairs gives high coverage with fewer test cases.

- Reduces the number of test cases significantly compared to exhaustive testing
 - Tools: PICT (Microsoft), AllPairs, TestNG DataProvider
 - Best for applications with many configurable parameters
-

Q25.

What is Use Case Testing?

Use Case Testing derives test cases from use cases, which describe how users interact with the system to achieve a goal. Each use case defines normal flow, alternate flows, and exception flows.

- Basic (Happy) Path — the main success scenario
- Alternate Flows — valid alternative ways to achieve the goal
- Exception Flows — error conditions and how the system handles them

Useful for end-to-end and integration testing from a user perspective.

Q26.

What is Error Guessing?

Error Guessing is an experience-based technique where testers use intuition and past experience to guess the most likely error-prone areas of the application and design test cases targeting those areas.

- Entering empty values or null inputs
- Entering very long strings in text fields
- Entering special characters where not expected
- Entering negative numbers for positive-only fields
- Clicking buttons multiple times rapidly

Tip: Often used alongside formal techniques to improve defect detection rates.

Section 4 – Test Planning & Strategy

Q27.

What is a Test Plan? What does it contain?

A Test Plan is a formal document that describes the scope, approach, resources, and schedule for testing activities. It is the master document that guides the entire testing effort.

Key contents:

- Test Objectives – What is being tested and goals
 - Scope (In-scope / Out-of-scope)
 - Test Strategy and Approach
 - Test Environment details
 - Entry and Exit Criteria
 - Test Schedule and milestones
 - Resource plan (team, tools)
 - Risk analysis and mitigation
 - Deliverables (test cases, reports, logs)
-

Q28.

What are Entry and Exit Criteria in testing?

Entry Criteria (conditions to START testing):

- Requirements documents are available and reviewed
- Test environment is set up and ready
- Test cases are reviewed and approved
- Build is deployed and smoke tested

Exit Criteria (conditions to STOP testing):

- All planned test cases are executed
 - Defect pass rate meets the defined threshold
 - No critical or high-severity open defects
 - Test summary report is prepared
-

Q29.

What is Risk-Based Testing?

Risk-Based Testing is an approach where test effort is prioritised based on the risk (likelihood × impact) of failure in different areas of the application.

- High-risk areas receive more testing attention and resources
- Helps manage time and budget constraints effectively
- Steps: Identify risks → Assess risk level → Prioritise test cases → Execute

Tip: Mention this when asked how you handle testing with tight deadlines.

Q30.

What is Traceability Matrix (RTM)?

A Requirements Traceability Matrix (RTM) is a document that maps and traces user requirements to test cases. It ensures that all requirements are covered by at least one test case and helps identify gaps.

Types:

- Forward Traceability – Requirements to Test Cases
- Backward Traceability – Test Cases to Requirements
- Bi-directional Traceability – Both directions

Columns typically include: Requirement ID, Requirement Description, Test Case ID(s), Test Status.

Q31.

What is Test Coverage? How do you achieve 100% coverage?

Test Coverage measures the degree to which the source code or requirements are tested. Types include:

- Statement Coverage – Every line of code executed
- Branch/Decision Coverage – Every branch of decision points
- Condition Coverage – Every boolean sub-expression
- Path Coverage – Every possible execution path
- Requirement Coverage – All requirements have test cases

Achieving 100% requirement coverage is the standard goal in functional testing. 100% code coverage is ideal but often impractical due to dead code or exception handlers.

Q32.

What is the difference between Test Strategy and Test Plan?

Test Strategy:

- High-level document defining the overall testing approach
- Organisation-level document, rarely changes
- Covers types of testing, tools, standards, and methodologies

Test Plan:

- Project-specific document derived from the test strategy
 - Changes per project and per release
 - Covers scope, schedule, resources, risks for a specific project
-

Section 5 – Defect / Bug Management

Q33.

What is a Defect? What is the difference between a Bug, Defect, Error, and Failure?

- Error – A human mistake made by a developer while writing code
- Bug/Defect – A flaw in the code resulting from an error, which can cause the application to behave incorrectly
- Failure – When the software does not perform its intended function in the live environment

Sequence: Error → Defect (in code) → Failure (in execution)

Q34.

What are the different Defect Severity levels?

- Critical – Application crashes, data loss, system unusable. Example: Login page not loading
 - High – Major feature broken, no workaround available. Example: Payment fails
 - Medium – Feature partially broken, workaround exists. Example: Export to PDF missing header
 - Low – Minor UI issues, cosmetic defects. Example: Typo in a label
-

Q35.

What is the difference between Severity and Priority?

Severity (technical impact):

- How badly the defect affects the application's functionality
- Set by the QA team based on technical impact

Priority (business urgency):

- How quickly the defect needs to be fixed
- Set by the product owner or business based on business impact

Examples:

- High Severity, Low Priority: A crash in a rarely used admin feature
 - Low Severity, High Priority: Company logo is wrong on the homepage
-

Q36.

What is the Defect Life Cycle?

The Defect Life Cycle (Bug Life Cycle) describes the various states a defect goes through from discovery to closure:

- New – Defect reported for the first time
 - Assigned – Assigned to a developer for fixing
 - Open – Developer acknowledges and starts working
 - Fixed – Developer has fixed the defect
 - Retest – QA re-tests to verify the fix
 - Verified – Fix confirmed by QA
 - Closed – Defect is resolved and closed
 - Rejected – Developer rejects it as not a defect
 - Deferred – Fix postponed to a future release
 - Reopened – Fix was inadequate; defect recurs
-

Q37.

What is a good bug report? What should it contain?

- Bug ID – Unique identifier
- Title/Summary – Clear, concise one-liner
- Environment – OS, browser, version, device
- Steps to Reproduce – Numbered, precise steps
- Expected Result – What should have happened
- Actual Result – What actually happened
- Severity and Priority
- Attachments – Screenshots, videos, logs
- Build/Version number

Tip: A good bug report allows any developer to reproduce the issue without asking for more information.

Q38.

What is a Showstopper defect?

A Showstopper (or Blocker) defect is a critical defect that prevents the testing team from proceeding with further testing of the application. It blocks the entire testing activity.

- Causes application to crash completely
- Prevents access to key functionalities
- Example: Unable to log in to the application

Testing is paused until the showstopper is fixed and a new build is released.

Section 6 – Test Execution & Reporting

Q39.

What is a Test Summary Report? What does it include?

A Test Summary Report is a document prepared at the end of a testing phase summarising testing activities and results.

- Test objectives and scope
 - Test environment details
 - Summary of test execution (total/pass/fail/blocked)
 - Defect summary (total, open, closed, deferred)
 - Test coverage achieved
 - Risks and issues encountered
 - Test team and sign-off
-

Q40.

What are the different test execution statuses?

- Pass – Test executed and actual result matches expected result
 - Fail – Actual result does not match expected result
 - Blocked – Cannot execute due to a dependency or showstopper
 - Not Run / Not Executed – Test not yet executed
 - In Progress – Test execution has started but not completed
 - Skipped – Intentionally skipped (deferred to next cycle)
 - N/A – Test case is not applicable in current context
-

Q41.

How do you prioritise test cases when time is limited?

- Execute smoke and sanity tests first
- Focus on high-priority and high-severity test cases
- Cover core/critical business flows (happy paths)
- Use risk-based testing to identify high-risk areas
- Prioritise recently changed or newly developed features
- Defer low-priority and edge-case tests to the next cycle

Tip: Always communicate with the project manager about what will and won't be covered.

Q42.

What is a Test Metrics? Give examples.

Test metrics are quantitative measures used to track and improve the quality of the testing process.

Examples:

- Test Case Pass Rate = $(\text{Passed} / \text{Total Executed}) \times 100$
 - Defect Density = $\text{Defects} / \text{Size of module (KLOC or function points)}$
 - Defect Removal Efficiency (DRE) = $\text{Defects found in testing} / \text{Total defects} \times 100$
 - Test Coverage = $(\text{Test cases executed} / \text{Total test cases}) \times 100$
 - Mean Time to Detect (MTTD)
 - Mean Time to Repair (MTTR)
-

Q43.

What is Re-testing vs Regression Testing?

Re-testing:

- Executing the specific test case(s) that previously failed after a bug fix
- Confirms that the reported defect has been fixed
- Only the failed test case is re-run

Regression Testing:

- Running previously passed test cases after any code change
 - Ensures existing functionality is not broken by the fix
 - Broader in scope than re-testing
-

Section 7 – Automation in Functional Testing

Q44.

When should you automate a test case?

- Test cases that are executed repeatedly (regression tests)
- High-volume data-driven tests
- Tests that are time-consuming to execute manually
- Stable features that rarely change
- Tests requiring precise timing or complex calculations

Do NOT automate:

- Tests that are executed only once or rarely
 - Exploratory or usability tests
 - Tests on unstable or frequently changing UI
-

Q45.

What are popular automation tools for Functional Testing?

- Selenium WebDriver – Web UI automation (Java, Python, C#, etc.)
 - Cypress – Modern JavaScript-based web testing
 - Playwright – Cross-browser automation by Microsoft
 - Appium – Mobile application testing (iOS and Android)
 - TestNG / JUnit – Test frameworks for Java
 - Cucumber (BDD) – Behaviour-Driven Development with Gherkin syntax
 - SoapUI / RestAssured – API functional testing
 - Katalon Studio – All-in-one automation tool
 - Robot Framework – Keyword-driven test automation
-

Q46.

What is Page Object Model (POM)?

Page Object Model (POM) is a design pattern in automation testing that creates a separate class (page object) for each web page. All web elements and interactions for that page are encapsulated in the corresponding class.

Benefits:

- Reduces code duplication
 - Improves code maintainability — element locator changes in one place only
 - Increases readability and reusability
 - Separates test logic from page interaction logic
-

Q47.

What is Behaviour-Driven Development (BDD)?

BDD is an approach where test scenarios are written in plain English (Gherkin syntax) that both technical and non-technical stakeholders can understand. It bridges communication between developers, testers, and business.

Gherkin keywords:

- Given – Precondition or initial context
- When – The action or event
- Then – Expected outcome
- And / But – Additional conditions

Example:

Given the user is on the login page

When they enter valid credentials and click Login

Then they should be redirected to the dashboard

Tools: Cucumber, SpecFlow, Behave

Q48.

What is the difference between White Box, Black Box, and Grey Box Testing?

Black Box Testing:

- No knowledge of internal code or structure
- Focuses on input/output behaviour
- Performed by QA testers

White Box Testing:

- Complete knowledge of internal code
- Focuses on code paths, branches, conditions
- Performed by developers

Grey Box Testing:

- Partial knowledge of internal structure
 - Tester has access to design docs and DB schemas
 - Combines black and white box approaches
-

Section 8 – Scenario-Based & Situational Questions

Q49.

How would you test a login page?

Positive Test Cases:

- Valid username and valid password — should log in successfully
- Login with email address format
- Login after password reset

Negative Test Cases:

- Invalid username, valid password — should show error
- Valid username, invalid password — should show error
- Empty username or empty password fields
- SQL injection in username/password fields
- XSS attack in input fields

Boundary & Edge Cases:

- Username/password at maximum character length
- Spaces before/after credentials
- Account locked after N failed attempts

UI/UX Cases:

- Password field shows asterisks
 - Remember Me checkbox functionality
 - Forgot Password link works
 - Session expires after inactivity
-

Q50.

How would you test an ATM machine?

Functional Test Cases:

- Valid card and correct PIN — successful login
- Invalid PIN — error message shown
- Account locked after 3 failed PIN attempts
- Cash withdrawal within account balance
- Cash withdrawal exceeding balance — insufficient funds message
- Withdrawal limit per transaction and per day
- Balance enquiry shows correct amount
- Fund transfer between accounts

Edge Cases:

- Withdrawing exact account balance
 - ATM runs out of cash
 - Network timeout during transaction
 - Card not ejected — timeout and card retention
 - Receipt not printed when paper is out
-

Q51.

How would you test a search functionality?

- Search with valid existing keyword — correct results returned
 - Search with non-existent keyword — 'No results' message
 - Search with partial keyword — relevant results shown
 - Search is case-insensitive (e.g., 'APPLE' = 'apple')
 - Search with special characters (!@#\$\$%) — handled gracefully
 - Search with very long text — no crash
 - Search with SQL injection or script tags — not vulnerable
 - Blank search — appropriate message or default results
 - Autocomplete / suggestions appear correctly
 - Search results are relevant and sorted correctly
 - Pagination of results works correctly
 - Performance: search returns results within acceptable time
-

Q52.

How would you handle a situation where requirements are unclear?

- Raise the ambiguity immediately to the BA or Product Owner
- Schedule a requirement clarification meeting
- Document all clarifications in writing (email or Jira comments)
- Use existing similar requirements or industry standards as a reference
- Write test cases based on best understanding and mark them for review
- Apply Error Guessing based on domain knowledge
- Escalate to the test manager if not resolved within the agreed time

Tip: Never start testing on unclear requirements without at least documented assumptions.

Q53.

What would you do if you find a critical bug one day before release?

- Immediately document the defect with all details and steps to reproduce
- Assign it the correct Severity (Critical) and Priority (High/Critical)
- Notify the development lead and test manager immediately
- Assess the impact — which features are affected?
- Discuss options: hotfix, rollback, workaround, defer to next release
- Run impact analysis — what else could be affected by the fix?
- Perform regression testing on the fix if a hotfix is applied
- Update stakeholders with clear and factual communication

Tip: Never hide a critical defect — transparency is essential.

Q54.

How do you ensure test cases cover all requirements?

- Create a Requirements Traceability Matrix (RTM)
 - Map every requirement to at least one test case
 - Participate in requirement review meetings
 - Use multiple design techniques (EP, BVA, Decision Tables)
 - Conduct peer reviews of test cases
 - Review test coverage reports regularly
 - Update RTM after every requirement change
-

Q55.

How do you test an e-commerce checkout flow?

Happy Path:

- Add items to cart → Proceed to checkout → Enter shipping details → Select payment → Place order → Confirmation email received

Functional Tests:

- Apply valid and invalid coupon codes
- Select different payment methods (credit card, UPI, COD)
- Change quantity in cart and verify price updates
- Remove item from cart
- Out-of-stock item cannot be added to cart

Edge Cases:

- Session expiry during checkout
 - Payment gateway timeout — order not duplicated
 - Back button during checkout doesn't cause issues
 - Address field validation (PIN code, special characters)
-

Q56.

What is the difference between Test Verification and Test Validation?

Verification:

- Confirms the software is built correctly per design docs
- Process: Reviews, walkthroughs, inspections
- Question answered: 'Are we building the product right?'

Validation:

- Confirms the software meets user needs
 - Process: Actual testing and execution
 - Question answered: 'Are we building the right product?'
-

Q57.

What is meant by Positive and Negative Testing?

Positive Testing:

- Validates the system works correctly with valid inputs
- Tests the 'happy path' or expected flow
- Example: Enter valid username and password → Login succeeds

Negative Testing:

- Validates the system handles invalid inputs gracefully
- Checks error messages and exception handling
- Example: Enter invalid password → Error message shown, no crash

Both types are essential for comprehensive functional coverage.

Q58.

What is Compatibility Testing?

Compatibility Testing verifies that the software works correctly across different browsers, operating systems, devices, screen resolutions, and network conditions.

Types:

- Browser Compatibility – Chrome, Firefox, Safari, Edge
 - OS Compatibility – Windows, macOS, Linux, iOS, Android
 - Device Compatibility – Desktop, tablet, mobile
 - Network Compatibility – 4G, 5G, WiFi, slow connections
 - Database Compatibility – Different DB versions
-

Q59.

What are the best practices in Functional Testing?

- Write test cases early based on requirements
 - Maintain traceability between requirements and test cases
 - Use a mix of testing techniques (EP, BVA, decision tables)
 - Include both positive and negative test cases
 - Keep test cases atomic — one test per condition
 - Write clear, reproducible, and independent test cases
 - Peer review all test cases before execution
 - Automate repetitive regression test cases
 - Log defects with clear steps, expected and actual results
 - Update test cases when requirements change
 - Always perform root cause analysis on critical defects
-

Q60.

What is Monkey Testing?

Monkey Testing is a type of testing where the tester or an automated tool provides random inputs to the application without any defined test cases, mimicking the behaviour of a monkey pressing random buttons.

Types:

- Dumb Monkey – Random inputs with no knowledge of the application
- Smart Monkey – Random inputs with some knowledge of valid actions
- Brilliant Monkey – Simulates user behaviour intelligently

It is useful for stress testing and finding unexpected crashes caused by unusual input combinations.

Bonus – Quick-Fire Definitions

Term	Definition
Alpha Testing	Testing performed by internal employees in a controlled environment before Beta release
Beta Testing	Testing by real end users in a production environment before final launch
Gorilla Testing	Exhaustively testing one module/feature from every angle
Fuzz Testing	Automated testing sending random, malformed data to find crashes and vulnerabilities
Mutation Testing	Modifying the code intentionally to check if test cases detect the changes
Static Testing	Testing without executing code — reviews, walkthroughs, inspections
Dynamic Testing	Testing by executing the software to observe behaviour
Positive Testing	Validating the system works with valid, expected inputs
Negative Testing	Validating the system handles invalid inputs gracefully
Test Harness	A collection of software and data configured to test a program unit by running it under varying conditions
Test Stub	A program simulating the behaviour of a lower-level module during integration testing (Top-Down)
Test Driver	A program simulating the behaviour of a higher-level module (Bottom-Up integration testing)
Defect Triage	A meeting to prioritise and assign defects, determining severity, priority, and fix timeline
SDLC	Software Development Life Cycle — the complete process of planning, creating, testing, and deployment
STLC	Software Testing Life Cycle — the phases of testing: planning, design, execution, and closure

Good Luck with Your Interview!

This guide covered: Fundamentals · Testing Types · Design Techniques · Test Planning · Defect Management · Test Execution · Automation · Scenario-Based Questions